

Identify Triangle Vertices

You are given a simple undirected graph G with n vertices, numbered 1 through n , and an array `edges` containing m edges. The i -th element of `edges`, (u_i, v_i) , indicates that there is an edge between the vertices u_i and v_i . A *triangle* in G is a group of three vertices in G where every pair of distinct vertices is connected by an edge.

Compute the number of vertices in G that are part of at least one triangle.

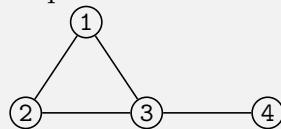
Example Test Cases:

Test Case 1

Input: $n = 4$, $m = 4$, `edges` = $[(1, 2), (2, 3), (1, 3), (3, 4)]$

Output: 3

Explanation:



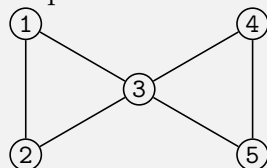
There is only one triangle in this graph: $(1, 2, 3)$. So, three vertices are part of a triangle.

Test Case 2

Input: $n = 5$, $m = 6$, `edges` = $[(1, 2), (2, 3), (1, 3), (3, 4), (4, 5), (3, 5)]$

Output: 5

Explanation:



There are two triangles in this graph: $(1, 2, 3)$ and $(3, 4, 5)$. So, each of the five vertices is part of at least one triangle.

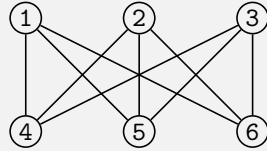
Test Case 3

Input: $n = 6$, $m = 9$,

`edges` = [(1,4), (1,5), (1,6), (2,4), (2,5), (2,6), (3,4), (3,5), (3,6)]

Output: 0

Explanation:



There are no triangles present in this graph.

Expected Time Complexity: $O(n^3)$

Input Format:

An integer n , an integer m , and an array `edges` of m pairs.

Output Format:

Return a single integer, the number of vertices belonging to at least one triangle, as defined above.

Constraints:

1. $1 \leq n \leq 400$
2. $0 \leq m \leq n(n-1)/2$
3. For all $1 \leq i \leq m$, $1 \leq u_i, v_i \leq n$, and there is an edge between u_i and v_i in G .
4. For all i , $u_i \neq v_i$ (G has no self-loops).
5. For all i, j with $i \neq j$, $\{u_i, v_i\} \neq \{u_j, v_j\}$ (G has no repeated/multiple edges)

Instructions:

You have been provided with two template files, containing the `main` and `solve` functions respectively. The `main` function, which has been completed for you, parses the input for each test case. For each test case, it calls the `solve` function, which must compute the number of vertices present in some triangle.

The `solve` function must compute this in $O(n^3)$ time.

(Refer to the `Instructions.pdf` file for detailed guidelines on the input-output format and additional test cases.)

Library Usage

For this lab, you are permitted to use any modules which are part of the standard library of either `C++` or `python`. For `C++`, do not add any extra `#include` directives; the provided `bits/stdc++.h` covers all allowed modules.